

## Objetivo

Crear un Deployment de Kubernetes y reconocer la relación entre Deployment, ReplicaSet y Pods. Escalar horizontalmente una aplicación utilizando enfoques imperativo y declarativo. Ejecutar un rolling update controlado y observar la transición entre ReplicaSets. Consultar el historial de revisiones de un Deployment y realizar un rollback hacia una versión anterior. Validar el estado final del workload mediante comandos kubectl orientados a operaciones reales.

## Requisitos previos

- Haber completado la Práctica 1 y la Práctica complementaria 1.1.
- Tener Docker Desktop instalado y en ejecución.
- Tener Minikube activo y kubectl configurado contra el clúster correcto.
- Utilizar Visual Studio Code con Git Bash como terminal integrada.
- Comprender la estructura básica de un manifiesto YAML de Kubernetes.
- Conocer los comandos kubectl get, kubectl describe y kubectl apply.

## Duración estimada

60 minutos

## Introducción

En esta práctica administrarás el ciclo de vida de un Deployment de Kubernetes. Crearás una aplicación web basada en NGINX, observarás el ReplicaSet y los Pods generados automáticamente, modificarás el número de réplicas, realizarás una actualización progresiva de imagen y finalmente ejecutarás un rollback. El objetivo es comprender cómo Kubernetes mantiene el estado deseado y cómo los Deployments permiten modificar aplicaciones sin administrar manualmente cada Pod.

## Instrucciones

### Preparación del directorio de trabajo

En esta práctica conservarás el directorio raíz del curso y crearás únicamente el subdirectorio `lab2`. También confirmarás que Minikube continúa disponible antes de administrar nuevos Deployments.

## Crear el subdirectorio de la práctica

- **Paso 1.** Abre **Docker Desktop** y confirma que el motor se encuentra activo, porque el nodo Minikube utiliza Docker como runtime del entorno local.
- **Paso 2.** Abre **Visual Studio Code**, selecciona **Git Bash** como terminal integrada y confirma que el directorio raíz del curso continúa disponible.
- **Paso 3.** Ejecuta el comando siguiente para crear el subdirectorio `lab2` dentro de la estructura existente del curso sin alterar los archivos de las prácticas anteriores.

```
mkdir -p /c/LABS/kubernetes/lab2
```

 Copiar

- **Paso 4.** Ejecuta el comando siguiente para cambiar la ubicación activa de Git Bash al directorio correspondiente al nuevo laboratorio.

```
cd /c/LABS/kubernetes/lab2
```

 Copiar

- **Paso 5.** Ejecuta `pwd` para verificar que la terminal se encuentra en la ruta correcta antes de crear los manifiestos de esta práctica.

```
pwd
```

 Copiar

### Salida esperada:

```
/c/LABS/kubernetes/lab2
```

 Copiar

- **Paso 6.** Ejecuta `minikube status` para confirmar que el clúster conserva sus componentes principales activos y puede recibir nuevas operaciones.

```
minikube status
```

 Copiar

- **Paso 7.** Ejecuta `kubectl get nodes` para comprobar que el nodo utilizado por el laboratorio aparece en estado `Ready`.

```
kubectl get nodes
```

 Copiar

### Resultado esperado:

El nodo `minikube` debe permanecer disponible y en estado `Ready` antes de continuar con la creación del Deployment.

**IMPORTANTE:** Esta práctica no requiere recrear Minikube ni utilizar varios nodos. El objetivo está centrado en el ciclo de vida del Deployment, no en scheduling o mantenimiento de nodos.

## Tarea 1. Crear un Deployment y explorar los recursos administrados

En esta tarea crearás un Deployment con tres réplicas de NGINX y observarás cómo Kubernetes genera automáticamente un ReplicaSet y los Pods necesarios para alcanzar el estado deseado.

### Tarea 1.1. Crear el manifiesto inicial

- **Paso 8.** Ejecuta el comando siguiente para crear el archivo `deployment-webapp.yaml`, donde conservarás la definición declarativa principal utilizada durante toda la práctica.

```
touch deployment-webapp.yaml
```

 Copiar

- **Paso 9.** Abre `deployment-webapp.yaml` desde el Explorador de VS Code y agrega el manifiesto siguiente para definir tres réplicas con estrategia RollingUpdate explícita.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
  labels:
    app: webapp
  annotations:
    kubernetes.io/change-cause: "Version inicial
con nginx:1.25"
spec:
  replicas: 3
  revisionHistoryLimit: 5
  selector:
    matchLabels:
      app: webapp
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: webapp
        version: v1
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
            limits:
              cpu: "100m"
              memory: "128Mi"
```

 Copiar

**NOTA:** `maxSurge: 1` permite crear como máximo un Pod adicional durante una actualización, mientras `maxUnavailable: 1` permite que como máximo uno de los Pods deseados no esté disponible temporalmente.

## Tarea 1.2. Validar y aplicar el Deployment

- **Paso 10.** Ejecuta una validación local para comprobar que la estructura del manifiesto es aceptada por kubectl antes de modificar el estado del clúster.

```
kubectl apply --dry-run=client -f deployment-webapp.yaml
```

[Copiar](#)

### Salida esperada:

```
deployment.apps/webapp created (dry run)
```

[Copiar](#)

- **Paso 11.** Ejecuta el comando siguiente para aplicar el manifiesto y permitir que Kubernetes comience a crear el ReplicaSet y los Pods declarados.

```
kubectl apply -f deployment-webapp.yaml
```

[Copiar](#)

- **Paso 12.** Espera a que Kubernetes confirme que las tres réplicas solicitadas se encuentran disponibles antes de continuar con las operaciones del Deployment.

```
kubectl rollout status deployment/webapp
```

[Copiar](#)

### Salida esperada:

```
deployment "webapp" successfully rolled out
```

[Copiar](#)

## Tarea 1.3. Explorar Deployment, ReplicaSet y Pods

- **Paso 13.** Ejecuta los comandos siguientes para observar de forma separada el Deployment, el ReplicaSet creado automáticamente y los Pods

administrados por ese ReplicaSet.

```
kubectl get deployment webapp
kubectl get replicaset -l app=webapp
kubectl get pods -l app=webapp -o wide
```

 Copiar

### Salida esperada aproximada:

```
NAME          READY    UP-TO-DATE    AVAILABLE    AGE
webapp        3/3      3              3             ...

NAME          DESIRED    CURRENT    READY    AGE
webapp-xxxxxxx 3          3          3        ...

NAME          READY    STATUS    RESTARTS
AGE
webapp-xxxxxxx-aaaaa 1/1      Running    0
...
webapp-xxxxxxx-bbbbb 1/1      Running    0
...
webapp-xxxxxxx-ccccc 1/1      Running    0
...
```

 Copiar

- **Paso 14.** Ejecuta `kubectl describe` para revisar la configuración activa, la estrategia RollingUpdate y los eventos generados durante la reconciliación.

```
kubectl describe deployment webapp
```

 Copiar

**Concepto clave:** El Deployment mantiene el estado deseado mediante un ReplicaSet. El ReplicaSet es quien conserva la cantidad de Pods requerida por la plantilla activa.

### Tarea finalizada

**Resultado esperado:** Deployment creado y administrado correctamente, identificando la relación entre Deployment, ReplicaSet y Pods.

## Prompt de apoyo para explicar lo realizado

➔ [Explicar la Tarea 1 con IA](#)

## Tarea 2. Escalar el Deployment de forma imperativa y declarativa

En esta tarea modificarás el número de réplicas para observar cómo Kubernetes crea o elimina Pods sin recrear el Deployment ni generar un nuevo ReplicaSet por un simple cambio de escala.

### Tarea 2.1. Escalar de tres a seis réplicas

- **Paso 15.** Ejecuta `kubectl scale` para modificar temporalmente el estado deseado del Deployment de tres a seis réplicas utilizando un comando imperativo.

```
kubectl scale deployment webapp --replicas=6
```

 Copiar

- **Paso 16.** Ejecuta el comando siguiente para observar cómo aparecen nuevos Pods hasta que el Deployment alcance las seis réplicas solicitadas.

```
kubectl get pods -l app=webapp -w
```

 Copiar

Cuando los seis Pods aparezcan en estado `Running`, presiona `Ctrl+C` para salir del modo de observación.

- **Paso 17.** Ejecuta los comandos siguientes para verificar que el mismo ReplicaSet ahora administra seis Pods sin haberse creado una nueva revisión de plantilla.

```
kubectl get deployment webapp  
kubectl get replicaset -l app=webapp
```

 Copiar

**Resultado esperado:**

El Deployment debe mostrar **6/6** réplicas disponibles y el ReplicaSet activo debe mostrar seis réplicas deseadas.

## Tarea 2.2. Reducir las réplicas desde el manifiesto

- **Paso 18.** Abre `deployment-webapp.yaml` en VS Code y modifica únicamente el campo `replicas` para establecer dos instancias como nuevo estado declarativo.

```
replicas: 2
```

 Copiar

- **Paso 19.** Guarda el archivo y valida nuevamente el manifiesto antes de aplicar la modificación declarativa al clúster.

```
kubectl apply --dry-run=client -f deployment-  
webapp.yaml
```

 Copiar

- **Paso 20.** Ejecuta `kubectl apply` para reconciliar el estado actual de seis Pods con las dos réplicas declaradas en el archivo.

```
kubectl apply -f deployment-webapp.yaml
```

 Copiar

- **Paso 21.** Ejecuta los comandos siguientes para confirmar que Kubernetes terminó los Pods excedentes y mantuvo únicamente dos réplicas disponibles.

```
kubectl get deployment webapp  
kubectl get pods -l app=webapp  
kubectl get replicaset -l app=webapp
```

 Copiar

### Salida esperada aproximada:

| NAME   | READY | UP-TO-DATE | AVAILABLE | AGE |
|--------|-------|------------|-----------|-----|
| webapp | 2/2   | 2          | 2         | ... |

 Copiar

**NOTA:** Cambiar solamente `spec.replicas` no modifica el Pod template, por lo que Kubernetes conserva el mismo ReplicaSet y ajusta únicamente su cantidad de réplicas.

### Tarea finalizada

**Resultado esperado:** Escalamiento horizontal ejecutado correctamente mediante enfoques imperativo y declarativo, ajustando las réplicas del Deployment.

#### Prompt de apoyo para explicar lo realizado

➤ [Explicar la Tarea 2 con IA](#)

## Tarea 3. Ejecutar un rolling update y observar la transición

En esta tarea actualizarás la imagen utilizada por el Deployment y observarás cómo Kubernetes crea un nuevo ReplicaSet mientras reduce progresivamente las réplicas del anterior.

### Tarea 3.1. Preparar el Deployment para observar la actualización

- **Paso 22.** Ejecuta el comando siguiente para escalar temporalmente el Deployment a cuatro réplicas y facilitar la observación del proceso de actualización progresiva.

```
kubectl scale deployment webapp --replicas=4
```

 Copiar

- **Paso 23.** Confirma que las cuatro réplicas actuales están disponibles antes de iniciar el cambio de versión de la imagen.

```
kubectl rollout status deployment/webapp  
kubectl get pods -l app=webapp
```

 Copiar

## Tarea 3.2. Actualizar la imagen del contenedor

- **Paso 24.** Ejecuta el comando siguiente para cambiar la imagen del contenedor `nginx` desde `nginx:1.25` hacia `nginx:1.26`, iniciando automáticamente un nuevo rollout.

```
kubectl set image deployment/webapp  
nginx=nginx:1.26
```

[Copiar](#)

- **Paso 25.** Registra una descripción de la modificación para que el historial del Deployment permita reconocer el motivo de esta nueva revisión.

```
kubectl annotate deployment webapp \  
  kubernetes.io/change-cause="Actualizacion a  
nginx:1.26" \  
  --overwrite
```

[Copiar](#)

- **Paso 26.** Ejecuta `kubectl rollout status` para observar el avance de la actualización hasta que las cuatro nuevas réplicas queden disponibles.

```
kubectl rollout status deployment/webapp
```

[Copiar](#)

### Salida esperada aproximada:

```
Waiting for deployment "webapp" rollout to finish:  
...  
deployment "webapp" successfully rolled out
```

[Copiar](#)

## Tarea 3.3. Comparar los ReplicaSets antes y después

- **Paso 27.** Ejecuta el comando siguiente para observar que ahora existen dos ReplicaSets: el anterior retenido con cero réplicas y el nuevo con las réplicas activas.

```
kubectl get replicaset -l app=webapp
```

[Copiar](#)

### Salida esperada aproximada:

| NAME             | DESIRED | CURRENT | READY | AGE |
|------------------|---------|---------|-------|-----|
| webapp-<hash-v1> | 0       | 0       | 0     | ... |
| webapp-<hash-v2> | 4       | 4       | 4     | ... |

 Copiar

- **Paso 28.** Ejecuta la consulta siguiente para comprobar directamente qué imagen utiliza cada Pod activo después del rolling update.

```
kubectl get pods -l app=webapp \
-o jsonpath='{range .items[*]}{.metadata.name}
{"\t"}{.spec.containers[0].image}{"\n"}{end}'
```

 Copiar

### Resultado esperado:

Todos los Pods activos deben utilizar `nginx:1.26`.

### Tarea 3.4. Sincronizar el manifiesto con el estado actual

- **Paso 29.** Abre `deployment-webapp.yaml` y actualiza los campos siguientes para que el archivo represente el estado que acabas de establecer en el clúster.

```
metadata:
  annotations:
    kubernetes.io/change-cause: "Actualizacion a
nginx:1.26"

spec:
  replicas: 4

  template:
    metadata:
      labels:
        app: webapp
        version: v2
    spec:
      containers:
        - name: nginx
          image: nginx:1.26
```

 Copiar

- **Paso 30.** Guarda el archivo y ejecuta una validación local para comprobar que la edición conserva una estructura válida antes de continuar.

```
kubectl apply --dry-run=client -f deployment-webapp.yaml
```

 Copiar

**IMPORTANTE:** No apliques todavía este archivo después de añadir `version: v2`. Cambiar una etiqueta dentro de `spec.template` genera otra revisión del Pod template. Para esta práctica basta con conservar el archivo preparado como referencia del estado deseado.

### Tarea finalizada

**Resultado esperado:** Rolling update completado hacia una nueva versión de NGINX, observando la creación y transición entre ReplicaSets.

#### Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 3 con IA](#)

## Tarea 4. Consultar revisiones y realizar rollback

En esta tarea revisarás las revisiones almacenadas por el Deployment y restaurarás la versión anterior para comprobar cómo Kubernetes reutiliza el ReplicaSet retenido.

### Tarea 4.1. Consultar el historial del Deployment

- **Paso 31.** Ejecuta el comando siguiente para listar las revisiones disponibles y revisar la causa registrada para cada cambio de plantilla.

```
kubectl rollout history deployment/webapp
```

 Copiar

**Salida esperada aproximada:**

```
deployment.apps/webapp
REVISION  CHANGE-CAUSE
1         Version inicial con nginx:1.25
2         Actualizacion a nginx:1.26
```

 Copiar

- **Paso 32.** Ejecuta los comandos siguientes para consultar individualmente las revisiones disponibles y comparar los valores conservados por Kubernetes.

```
kubectl rollout history deployment/webapp --
revision=1
kubectl rollout history deployment/webapp --
revision=2
```

 Copiar

## Tarea 4.2. Ejecutar rollback hacia la versión anterior

- **Paso 33.** Ejecuta `kubectl rollout undo` para solicitar que Kubernetes restaure la revisión inmediatamente anterior del Deployment.

```
kubectl rollout undo deployment/webapp
```

 Copiar

- **Paso 34.** Ejecuta el comando siguiente para seguir el proceso de reversión hasta que Kubernetes confirme que el Deployment volvió a quedar disponible.

```
kubectl rollout status deployment/webapp
```

 Copiar

### Salida esperada:

```
deployment "webapp" successfully rolled out
```

 Copiar

- **Paso 35.** Ejecuta la consulta siguiente para confirmar que los Pods activos volvieron a utilizar la imagen correspondiente a la versión inicial.

```
kubectl get pods -l app=webapp \
-o jsonpath='{range .items[*]}{.metadata.name}
{"\t"}{.spec.containers[0].image}{"\n"}{end}'
```

 Copiar

### Resultado esperado:

Los Pods activos deben utilizar nuevamente:

```
nginx:1.25
```

 Copiar

## Tarea 4.3. Observar la reutilización de ReplicaSets

- **Paso 36.** Ejecuta el comando siguiente para comparar los ReplicaSets después del rollback y comprobar cuál volvió a recibir réplicas activas.

```
kubectl get replicaset -l app=webapp
```

 Copiar

- **Paso 37.** Ejecuta nuevamente el historial para observar cómo Kubernetes registra la reversión dentro de la secuencia de revisiones.

```
kubectl rollout history deployment/webapp
```

 Copiar

**Concepto clave:** El rollback no requiere reconstruir manualmente los Pods anteriores. El Deployment conserva ReplicaSets históricos según `revisionHistoryLimit` y puede volver a escalar la plantilla correspondiente.

### Tarea finalizada

**Resultado esperado:** Historial de revisiones consultado y rollback ejecutado correctamente, restaurando la versión anterior del Deployment.

## Prompt de apoyo para explicar lo realizado

➤ [Explicar la Tarea 4 con IA](#)

### ✓ Tarea 5. Validar el ciclo de vida completo del Deployment

En esta tarea comprobarás que el Deployment quedó operativo después de las operaciones de escalamiento, actualización y rollback, relacionando el estado final con los ReplicaSets conservados.

#### Tarea 5.1. Revisar el estado general

- **Paso 38.** Ejecuta el comando siguiente para mostrar el Deployment, sus ReplicaSets y los Pods activos asociados con `app=webapp`.

```
kubectl get deployment webapp
kubectl get replicaset -l app=webapp
kubectl get pods -l app=webapp -o wide
```

 Copiar

- **Paso 39.** Ejecuta `kubectl rollout status` para confirmar que no existe ninguna actualización pendiente y que el Deployment se encuentra estable.

```
kubectl rollout status deployment/webapp
```

 Copiar

#### Tarea 5.2. Ejecutar la verificación final

- **Paso 40.** Copia y ejecuta el bloque completo para validar automáticamente la disponibilidad, cantidad de réplicas, imagen activa, historial y ReplicaSets del Deployment.

```
echo "=== Verificacion final de la Practica 1.1
=== "

READY=$(kubectl get deployment webapp \
  -o jsonpath='{.status.readyReplicas}')

DESIRED=$(kubectl get deployment webapp \
  -o jsonpath='{.spec.replicas}')

if [ "$READY" = "$DESIRED" ]; then
  echo "✅ Deployment disponible:
$READY/$DESIRED replicas"
else
  echo "❌ Deployment incompleto:
$READY/$DESIRED replicas"
fi

IMAGE=$(kubectl get deployment webapp \
  -o
jsonpath='{.spec.template.spec.containers[0].ima
ge}')

if [ "$IMAGE" = "nginx:1.25" ]; then
  echo "✅ Rollback confirmado: imagen activa
nginx:1.25"
else
  echo "❌ Imagen inesperada: $IMAGE"
fi

RS_COUNT=$(kubectl get rs -l app=webapp --no-
headers | wc -l)

if [ "$RS_COUNT" -ge "2" ] 2>/dev/null; then
  echo "✅ Existen ReplicaSets historicos para
el Deployment"
else
  echo "❌ No se identificaron ReplicaSets
historicos"
fi

echo ""
echo "Historial de revisiones:"
kubectl rollout history deployment/webapp
```

 Copiar

```
echo ""  
echo "ReplicaSets:"  
kubectl get rs -l app=webapp  
  
echo "=== Fin de verificacion ==="
```

### Salida esperada aproximada:

```
=== Verificacion final de la Practica 2 ===  
✓ Deployment disponible: 4/4 replicas  
✓ Rollback confirmado: imagen activa nginx:1.25  
✓ Existen ReplicaSets historicos para el Deployment
```

[Copiar](#)

Historial de revisiones:

deployment.apps/webapp

REVISION CHANGE-CAUSE

...

ReplicaSets:

| NAME | DESIRED | CURRENT | READY | AGE |
|------|---------|---------|-------|-----|
|------|---------|---------|-------|-----|

...

=== Fin de verificacion ===

**IMPORTANTE:** No elimines todavía el Deployment `webapp`. La siguiente práctica complementaria utilizará estos recursos para profundizar en labels, selectors y ReplicaSets.

### Tarea finalizada

**Resultado esperado:** Deployment estable y validado después del ciclo completo de escalamiento, actualización y reversión.

### Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 5 con IA](#)

## 🔧 Resolución de problemas

### Problema 1. El Deployment no alcanza todas las réplicas disponibles

**Síntoma:** `kubectl get deployment webapp` muestra menos réplicas disponibles que las definidas en `DESIRED`.

**Causa probable:** Alguno de los Pods no puede iniciar, la imagen no está disponible o el nodo no tiene recursos suficientes para ejecutar todas las réplicas solicitadas.

#### Solución:

Identifica primero los Pods con problemas y revisa sus eventos antes de modificar el Deployment.

```
kubectl get pods -l app=webapp
POD_NAME=$(kubectl get pods -l app=webapp \
  -o jsonpath='{.items[0].metadata.name}')
kubectl describe pod "$POD_NAME"
```

[Copiar](#)

Revisa principalmente la sección `Events` para identificar errores de descarga de imagen, scheduling o recursos.

### Problema 2. El rolling update queda detenido

**Síntoma:** `kubectl rollout status deployment/webapp` permanece esperando y uno o más Pods nuevos no alcanzan estado `Running`.

**Causa probable:** La nueva imagen no puede descargarse, el contenedor falla al iniciar o la actualización no puede cumplir temporalmente con la estrategia configurada.

#### Solución:

Consulta el estado de los Pods y los eventos del Deployment para identificar el punto exacto donde se detuvo la actualización.

```
kubectl get pods -l app=webapp
kubectl describe deployment webapp
```

[Copiar](#)

Si el problema está relacionado con la nueva imagen y necesitas recuperar rápidamente la versión estable, ejecuta:

```
kubectl rollout undo deployment/webapp  
kubectl rollout status deployment/webapp
```

 Copiar

### Problema 3. El historial muestra CHANGE-CAUSE como

**Síntoma:** `kubectl rollout history deployment/webapp` muestra una revisión sin descripción en la columna `CHANGE-CAUSE`.

**Causa probable:** La anotación `kubernetes.io/change-cause` no estaba definida en el Deployment cuando se creó esa revisión o fue registrada después de generar el cambio.

#### Solución:

Comprueba primero las anotaciones actuales del Deployment.

```
kubectl get deployment webapp \n-o jsonpath='{.metadata.annotations}'
```

 Copiar

Para futuras revisiones, registra la causa antes o junto con el cambio que modificará el Pod template.

### Problema 4. El rollback no devuelve la imagen esperada

**Síntoma:** Después de `kubectl rollout undo`, la imagen activa no corresponde con `nginx:1.25`.

**Causa probable:** Existen más revisiones de las previstas debido a modificaciones adicionales realizadas durante la práctica.

#### Solución:

Consulta el historial y selecciona explícitamente la revisión que contiene la versión requerida.

```
kubectl rollout history deployment/webapp
kubectl rollout history deployment/webapp --
revision=1
kubectl rollout undo deployment/webapp --to-
revision=1
kubectl rollout status deployment/webapp
```

 Copiar

Después confirma nuevamente la imagen activa:

```
kubectl get deployment webapp \
-o
jsonpath='{.spec.template.spec.containers[0].image}
{"\n"}'
```

 Copiar

## Problema 5. El número de réplicas cambia inesperadamente al aplicar el manifiesto

**Síntoma:** Después de utilizar `kubectl scale`, al ejecutar posteriormente `kubectl apply -f deployment-webapp.yaml`, el número de Pods vuelve al valor definido dentro del archivo.

**Causa probable:** `kubectl scale` modificó el estado actual del clúster, pero el manifiesto local conserva otro valor en `spec.replicas`.

### Solución:

Decide cuál debe ser el estado deseado definitivo y sincroniza el campo `replicas` del manifiesto antes de volver a aplicarlo.

```
grep -n "replicas:" deployment-webapp.yaml
kubectl get deployment webapp \
-o jsonpath='{.spec.replicas}{"\n"}'
```

 Copiar

La diferencia entre ambos valores demuestra por qué es importante mantener sincronizados el estado declarativo y las operaciones imperativas.

## ¡Felicidades! Has completado el Laboratorio.

### Resultado final

Al finalizar la práctica tendrás un Deployment administrado mediante manifiestos YAML, habrás escalado sus réplicas, ejecutado un rolling update entre dos versiones de NGINX, consultado el historial de revisiones y restaurado una versión anterior mediante rollback, verificando en cada etapa la relación entre Deployment, ReplicaSets y Pods.

### Notas y consideraciones

- Esta práctica reutiliza el clúster Minikube creado anteriormente, pero utiliza recursos nuevos para evitar modificar la aplicación Node.js del Lab 1.
- Los labels y selectors se utilizan únicamente en el nivel necesario para operar el Deployment. Su análisis detallado se realizará en la práctica complementaria siguiente.
- No elimines el clúster Minikube al finalizar; se reutilizará durante las siguientes prácticas.
- Durante un rolling update es normal observar temporalmente Pods de dos ReplicaSets diferentes.

### URL de referencia

- [Deployments](#)
- [Performing a Rolling Update](#)
- [kubectl rollout](#)
- [kubectl scale](#)